

Tutorial de Cursores en PL/pgSQL

<https://gemini.google.com/app/0c9c73d236c43237>

User prompt: utilizando esta base de datos: CREATE SCHEMA tienda; SET search_path TO tienda; CREATE TABLE clientes (cliente_id SERIAL PRIMARY KEY, nombre TEXT NOT NULL, email TEXT UNIQUE NOT NULL, fecha_alta DATE NOT NULL DEFAULT CURRENT_DATE); CREATE TABLE productos (producto_id SERIAL PRIMARY KEY, nombre TEXT NOT NULL, precio NUMERIC(10,2) NOT NULL, stock INTEGER NOT NULL DEFAULT 0, activo BOOLEAN NOT NULL DEFAULT TRUE); CREATE TABLE pedidos (pedido_id SERIAL PRIMARY KEY, cliente_id INTEGER NOT NULL REFERENCES clientes(cliente_id), fecha_pedido TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP, estado TEXT NOT NULL DEFAULT 'pendiente'); CREATE TABLE lineas_pedido (linea_id SERIAL PRIMARY KEY, pedido_id INTEGER NOT NULL REFERENCES pedidos(pedido_id), producto_id INTEGER NOT NULL REFERENCES productos(producto_id), cantidad INTEGER NOT NULL, precio_unit NUMERIC(10,2) NOT NULL); CREATE TABLE pagos (pago_id SERIAL PRIMARY KEY, pedido_id INTEGER NOT NULL REFERENCES pedidos(pedido_id), fecha_pago TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP, importe NUMERIC(10,2) NOT NULL, metodo TEXT NOT NULL); CREATE TABLE empleados (id SERIAL PRIMARY KEY, nombre TEXT NOT NULL, salario NUMERIC(10,2) NOT NULL, departamento TEXT, fecha_contratacion DATE DEFAULT CURRENT_DATE); INSERT INTO empleados (nombre, salario, departamento) VALUES ('Ana Torres', 2800.00, 'Ventas'), ('Luis Martínez', 4500.00, 'IT'), ('Carla Gómez', 5200.00, 'IT'), ('Dmitry Komarov', 3100.00, 'Contabilidad'), ('James Smith', 7000.00, 'Dirección'), ('Mary Sexton', 3900.00, 'Marketing'), ('Agnes Miller', 6100.00, 'IT'), ('Chun Yen', 2950.00, 'Ventas'); INSERT INTO clientes (nombre, email) VALUES ('Juan Pérez', 'juan@example.com'), ('María López', 'maria@example.com'); INSERT INTO productos (nombre, precio, stock) VALUES ('Teclado mecánico', 79.90, 50), ('Ratón inalámbrico', 39.50, 100), ('Monitor 27"', 249.00, 20); INSERT INTO pedidos (cliente_id, estado) VALUES (1, 'pendiente'), (2, 'pendiente'); INSERT INTO lineas_pedido (pedido_id, producto_id, cantidad, precio_unit) VALUES (1, 1, 1, 79.90), (1, 2, 2, 39.50), (2, 3, 1, 249.00); INSERT INTO pagos (pedido_id, importe, metodo) VALUES (1, 158.90, 'tarjeta'), (2, 249.00, 'transferencia'); creame un tutorial completo de cursores en plpgsql con postgresql 18 d, ponme ejemplos con enunciado y resultado. todo esto en plpgsql, con postgresql 18. no te olvides de crear ejemplos con variables out y escribelas con este formato: v_linead_id, v_pedido_id, ... etc. Todas las variables que se declaren tanto con out como con return table, etc deben tener este formato: v_linead_id, v_pedido_id, ... etc. Pon tambien ejmplos de cursores explicitos e implicitos con variables out y sin ellas. cursores con return table con record y sin el, con cursores explicitos e implicitos.

Response: Este tutorial cubre el uso de **cursores en PL/pgSQL** para PostgreSQL 18, utilizando el esquema `tienda` provisto.

Tipos de Cursores en PL/pgSQL

- **Cursores Implícitos (FOR r IN SELECT ...):** PostgreSQL maneja la apertura, lectura (*fetch*) y cierre de manera automática. Es la opción más eficiente y limpia para la mayoría de los casos.
- **Cursores Explícitos (refcursor / CURSOR FOR):** Se declaran, abren (OPEN), leen (FETCH) y cierran (CLOSE) manualmente. Permiten mayor control sobre el avance fila por fila.

1. Cursores Implícitos

Ejemplo 1.1: Cursor implícito con RETURN TABLE (Usando un tipo RECORD)

Enunciado: Crea una función que devuelva el listado de empleados del departamento de 'IT' junto con un mensaje formateado con su salario.

```
CREATE OR REPLACE FUNCTION tienda.fn_empleados_it()
RETURNS TABLE (
    v_id INTEGER,
    v_nombre TEXT,
    v_info TEXT
) AS $$
DECLARE
    v_rec RECORD; -- Variable tipo RECORD para capturar las filas del cursor implícito
BEGIN
    FOR v_rec IN
        SELECT id, nombre, salario
        FROM tienda.empleados
        WHERE departamento = 'IT'
        ORDER BY id
    LOOP
        v_id := v_rec.id;
        v_nombre := v_rec.nombre;
        v_info := 'Salario actual: ' || v_rec.salario || ' €';
        RETURN NEXT; -- Devuelve la fila construida en cada iteración
    END LOOP;
END;
$$ LANGUAGE plpgsql;

-- Ejecución:
SELECT * FROM tienda.fn_empleados_it();
```

Resultado:

v_id	v_nombre	v_info
2	Luis Martínez	Salario actual: 4500.00 €
3	Carla Gómez	Salario actual: 5200.00 €
7	Agnes Miller	Salario actual: 6100.00 €

Ejemplo 1.2: Cursor implícito con RETURN TABLE (Sin usar RECORD, asignación directa)

Enunciado: Obtener la lista de productos con stock superior a 20 asignando directamente las columnas de la consulta a las variables de retorno.

```
CREATE OR REPLACE FUNCTION tienda.fn_productos_stock()
RETURNS TABLE (
    v_producto_id INTEGER,
    v_nombre TEXT,
    v_stock INTEGER
) AS $$
BEGIN
    -- Se iteran directamente las variables de salida definidas en RETURN TABLE
    FOR v_producto_id, v_nombre, v_stock IN
        SELECT producto_id, nombre, stock
        FROM tienda.productos
        WHERE stock > 20
    LOOP
        RETURN NEXT;
    END LOOP;
END;
$$ LANGUAGE plpgsql;

-- Ejecución:
SELECT * FROM tienda.fn_productos_stock();
```

Resultado:

v_producto_id	v_nombre	v_stock
1	Teclado mecánico	50
2	Ratón inalámbrico	100

Ejemplo 1.3: Cursor implícito con parámetros OUT

Enunciado: Dado el ID de un pedido, calcular mediante un cursor implícito el total consumido (suma de cantidad * precio_unit) y la cantidad total de artículos.

```
CREATE OR REPLACE FUNCTION tienda.fn_resumen_pedido(
    IN v_pedido_id INTEGER,
    OUT v_total_importe NUMERIC(10,2),
    OUT v_total_articulos INTEGER
) AS $$
DECLARE
    v_rec RECORD;
BEGIN
    v_total_importe := 0;
    v_total_articulos := 0;

    FOR v_rec IN
        SELECT cantidad, precio_unit
        FROM tienda.lineas_pedido
        WHERE pedido_id = v_pedido_id
    LOOP
        v_total_importe := v_total_importe + (v_rec.cantidad * v_rec.precio_unit);
        v_total_articulos := v_total_articulos + v_rec.cantidad;
    END LOOP;
END;
$$ LANGUAGE plpgsql;

-- Ejecución:
SELECT * FROM tienda.fn_resumen_pedido(1);
```

Resultado:

v_total_importe	v_total_articulos
158.90	3

2. Cursores Explícitos

Ejemplo 2.1: Cursor explícito básico con lectura manual (OPEN, FETCH, CLOSE)

Enunciado: Crear una función que reciba el nombre de un departamento y devuelva la suma de los salarios de sus dos empleados mejor pagados recorriéndolos secuencialmente con un cursor explícito.

```
CREATE OR REPLACE FUNCTION tienda.fn_top2_salarios_dep(
    IN v_departamento TEXT,
    OUT v_suma_salarios NUMERIC(10,2)
) AS $$
DECLARE
    -- Declaración del cursor explícito
```

```

v_cursor CURSOR FOR
    SELECT salario
    FROM tienda.empleados
    WHERE departamento = v_departamento
    ORDER BY salario DESC;

v_salario NUMERIC(10,2);
v_contador INTEGER := 0;
BEGIN
    v_suma_salarios := 0;

    OPEN v_cursor;
    LOOP
        FETCH v_cursor INTO v_salario;
        EXIT WHEN NOT FOUND OR v_contador = 2; -- Condición de parada

        v_suma_salarios := v_suma_salarios + v_salario;
        v_contador := v_contador + 1;
    END LOOP;
    CLOSE v_cursor;
END;
$$ LANGUAGE plpgsql;

-- Ejecución:
SELECT tienda.fn_top2_salarios_dep('IT');
```

Resultado:

```

fn_top2_salarios_dep
-----
                11300.00
```

Ejemplo 2.2: Cursor explícito con RETURN TABLE y RECORD

Enunciado: Recorrer las líneas de pedido registradas utilizando un cursor explícito y devolverlas formateadas junto al cálculo del subtotal de cada línea.

```

CREATE OR REPLACE FUNCTION tienda.fn_detalle_lineas_pedido()
RETURNS TABLE (
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_subtotal NUMERIC(10,2)
) AS $$
DECLARE
    v_cur_lineas CURSOR FOR
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido;
    v_reg RECORD;
BEGIN
    OPEN v_cur_lineas;
    LOOP
        FETCH v_cur_lineas INTO v_reg;
        EXIT WHEN NOT FOUND;

        v_linea_id := v_reg.linea_id;
        v_pedido_id := v_reg.pedido_id;
        v_producto_id := v_reg.producto_id;
        v_subtotal := v_reg.cantidad * v_reg.precio_unit;

        RETURN NEXT;
    END LOOP;
    CLOSE v_cur_lineas;
END;
$$ LANGUAGE plpgsql;

-- Ejecución:
SELECT * FROM tienda.fn_detalle_lineas_pedido();
```

Resultado:

v_linea_id	v_pedido_id	v_producto_id	v_subtotal
1	1	1	79.90
2	1	2	79.00
3	2	3	249.00

Ejemplo 2.3: Cursor explícito parametrizado con RETURN TABLE (Sin RECORD)

Enunciado: Crear un cursor explícito que reciba como parámetro un método de pago y devuelva los pagos registrados asignando los valores directamente a variables individuales sin usar un tipo RECORD.

```
CREATE OR REPLACE FUNCTION tienda.fn_pagos_por_metodo(IN p_metodo TEXT)
RETURNS TABLE (
    v_pago_id INTEGER,
    v_pedido_id INTEGER,
    v_importe NUMERIC(10,2)
) AS $$
DECLARE
    -- Cursor explícito parametrizado
    v_cur_pagos CURSOR (p_met TEXT) FOR
        SELECT pago_id, pedido_id, importe
        FROM tienda.pagos
        WHERE metodo = p_met;
BEGIN
    OPEN v_cur_pagos(p_metodo);
    LOOP
        -- Asignación directa a las variables del RETURN TABLE
        FETCH v_cur_pagos INTO v_pago_id, v_pedido_id, v_importe;
        EXIT WHEN NOT FOUND;

        RETURN NEXT;
    END LOOP;
    CLOSE v_cur_pagos;
END;
$$ LANGUAGE plpgsql;

-- Ejecución:
SELECT * FROM tienda.fn_pagos_por_metodo('tarjeta');
```

Resultado:

v_pago_id	v_pedido_id	v_importe
1	1	158.90

Ejemplo 2.4: Cursor explícito con múltiples variables OUT

Enunciado: Obtener mediante variables OUT la información básica del cliente que ha registrado la fecha de alta más reciente en la tienda usando un cursor explícito.

```
CREATE OR REPLACE FUNCTION tienda.fn_ultimo_cliente_registrado(
    OUT v_cliente_id INTEGER,
    OUT v_nombre TEXT,
    OUT v_email TEXT
) AS $$
DECLARE
    v_cur_cliente CURSOR FOR
        SELECT cliente_id, nombre, email
        FROM tienda.clientes
        ORDER BY fecha_alta DESC, cliente_id DESC
        LIMIT 1;
BEGIN
    OPEN v_cur_cliente;
    FETCH v_cur_cliente INTO v_cliente_id, v_nombre, v_email;
    CLOSE v_cur_cliente;
END;
$$ LANGUAGE plpgsql;

-- Ejecución:
SELECT * FROM tienda.fn_ultimo_cliente_registrado();
```

Resultado:

v_cliente_id	v_nombre	v_email
2	María López	maria@example.com

Resumen de Buenas Prácticas

- Prioriza cursores implícitos (FOR rec IN SELECT ...):** Tienen mejor rendimiento interna en PostgreSQL, no requieren gestionar la apertura o el cierre explícito y evitan pérdidas de memoria por cursores no cerrados.
- Usa cursores explícitos solo cuando sea necesario:** Principalmente cuando requieras navegación condicional, procesamiento por bloques parciales o pasar un puntero `refcursor` a una aplicación cliente o función externa.
- Liberación de recursos:** Asegúrate de que todo cursor explícito abierto con `OPEN` finalice con un comando `CLOSE` dentro del mismo bloque de ejecución.

User prompt: ahora haz lo mismo pero utiliza RETURNS SETOF RECORD, %ROWTYPE, y return query.

Response: En este bloque completamos el tutorial agregando el uso de **RETURNS SETOF RECORD**, variables de tipo **%ROWTYPE** y la sentencia **RETURN QUERY** (aplicada tanto a cursores implícitos como explícitos).

Continuamos con el estándar estricto de nomenclaturas (**v_id**, **v_nombre**, etc.) para todas las variables declaradas y los campos devueltos.

3. Uso de RETURNS SETOF RECORD y %ROWTYPE

Cuando una función utiliza **RETURNS SETOF RECORD**, se debe especificar la definición de tipos/columnas (cláusula **AS (...)**) al invocar la función desde la consulta **SELECT**.

Ejemplo 3.1: Cursor Implícito con RETURNS SETOF RECORD y %ROWTYPE

Enunciado: Crear una función que recorra la tabla **productos** usando un cursor implícito sobre una variable declarada con el tipo de la tabla (**tienda.productos%ROWTYPE**) y devuelva un conjunto de registros desestructurados.

```
CREATE OR REPLACE FUNCTION tienda.fn_productos_setof_record()
RETURNS SETOF RECORD AS $$
DECLARE
    -- Variable basada en la estructura exacta de la tabla productos
    v_producto_rec tienda.productos%ROWTYPE;

    -- Variables para construir la salida
    v_producto_id INTEGER;
    v_nombre TEXT;
    v_precio NUMERIC(10,2);
    v_stock INTEGER;
BEGIN
    FOR v_producto_rec IN
        SELECT *
        FROM tienda.productos
        ORDER BY producto_id
    LOOP
        v_producto_id := v_producto_rec.producto_id;
        v_nombre      := v_producto_rec.nombre;
        v_precio      := v_producto_rec.precio;
        v_stock       := v_producto_rec.stock;

        -- Retorna la tupla construida con las variables locales
        RETURN NEXT (v_producto_id, v_nombre, v_precio, v_stock);
    END LOOP;
END;
$$ LANGUAGE plpgsql;

-- Ejecución (es obligatorio definir la estructura de la salida con AS):
SELECT * FROM tienda.fn_productos_setof_record()
AS (
    v_producto_id INTEGER,
    v_nombre TEXT,
    v_precio NUMERIC(10,2),
    v_stock INTEGER
);
```

Resultado:

v_producto_id	v_nombre	v_precio	v_stock
1	Teclado mecánico	79.90	50
2	Ratón inalámbrico	39.50	100
3	Monitor 27"	249.00	20

Ejemplo 3.2: Cursor Explícito con RETURNS SETOF RECORD y %ROWTYPE

Enunciado: Recorrer los empleados de un departamento específico utilizando un cursor explícito y una variable de tipo **tienda.empleados%ROWTYPE**, devolviendo los registros mediante **SETOF RECORD**.

```
CREATE OR REPLACE FUNCTION tienda.fn_empleados_dep_setof(IN p_dep TEXT)
RETURNS SETOF RECORD AS $$
DECLARE
    v_cur_emp CURSOR FOR
        SELECT *
        FROM tienda.empleados
        WHERE departamento = p_dep;

    v_emp_rec tienda.empleados%ROWTYPE;

    v_id INTEGER;
```

```

v_nombre TEXT;
v_salario NUMERIC(10,2);
v_departamento TEXT;
BEGIN
OPEN v_cur_emp;
LOOP
    FETCH v_cur_emp INTO v_emp_rec;
    EXIT WHEN NOT FOUND;

    v_id          := v_emp_rec.id;
    v_nombre      := v_emp_rec.nombre;
    v_salario     := v_emp_rec.salario;
    v_departamento := v_emp_rec.departamento;

    RETURN NEXT (v_id, v_nombre, v_salario, v_departamento);
END LOOP;
CLOSE v_cur_emp;
END;
$$ LANGUAGE plpgsql;

-- Ejecución:
SELECT * FROM tienda.fn_empleados_dep_setof('Ventas')
AS (
    v_id INTEGER,
    v_nombre TEXT,
    v_salario NUMERIC(10,2),
    v_departamento TEXT
);

```

Resultado:

v_id	v_nombre	v_salario	v_departamento
1	Ana Torres	2800.00	Ventas
8	Chun Yen	2950.00	Ventas

4. Uso de RETURN QUERY

RETURN QUERY permite volcar de forma masiva los resultados de una consulta o de un cursor sin necesidad de iterar fila por fila mediante un bucle LOOP.

Ejemplo 4.1: RETURN QUERY con Consulta Directa (Cursor Implícito Optimizado)

Enunciado: Devolver la tabla de clientes formateada usando RETURN QUERY y RETURN TABLE.

```

CREATE OR REPLACE FUNCTION tienda.fn_listar_clientes_rq()
RETURNS TABLE (
    v_cliente_id INTEGER,
    v_nombre TEXT,
    v_email TEXT,
    v_fecha_alta DATE
) AS $$
BEGIN
    -- RETURN QUERY ejecuta la consulta interna e inyecta todas las filas al conjunto de salida
    RETURN QUERY
    SELECT cliente_id, nombre, email, fecha_alta
    FROM tienda.clientes
    ORDER BY cliente_id;
END;
$$ LANGUAGE plpgsql;

-- Ejecución:
SELECT * FROM tienda.fn_listar_clientes_rq();

```

Resultado:

v_cliente_id	v_nombre	v_email	v_fecha_alta
1	Juan Pérez	juan@example.com	2026-03-20
2	María López	maria@example.com	2026-03-20

Ejemplo 4.2: RETURN QUERY EXECUTE (Consulta Dinámica)

Enunciado: Filtrar los pagos según el método indicado pasando parámetros dinámicos a RETURN QUERY EXECUTE.

```

CREATE OR REPLACE FUNCTION tienda.fn_pagos_dinamico_rq(IN p_metodo TEXT)
RETURNS TABLE (
    v_pago_id INTEGER,
    v_pedido_id INTEGER,

```

```

    v_importe NUMERIC(10,2),
    v_metodo TEXT
) AS $$
DECLARE
    v_sql TEXT;
BEGIN
    v_sql := 'SELECT pago_id, pedido_id, importe, metodo FROM tienda.pagos WHERE metodo = $1';

    RETURN QUERY EXECUTE v_sql USING p_metodo;
END;
$$ LANGUAGE plpgsql;

-- Ejecución:
SELECT * FROM tienda.fn_pagos_dinamico_rq('transferencia');
```

Resultado:

v_pago_id	v_pedido_id	v_importe	v_metodo
2	2	249.00	transferencia

Ejemplo 4.3: RETURN QUERY en combinación con un Cursor Explícito (Puntero refcursor)

Enunciado: Abrir un cursor explícito (refcursor), consumirlo parcialmente y volcar los resultados filtrados mediante RETURN QUERY.

```

CREATE OR REPLACE FUNCTION tienda.fn_empleados_altos_salarios_rq(IN p_salario_min NUMERIC(10,2))
RETURNS TABLE (
    v_id INTEGER,
    v_nombre TEXT,
    v_salario NUMERIC(10,2),
    v_departamento TEXT
) AS $$
DECLARE
    -- Declaración de una variable refcursor no vinculada
    v_cur_ref refcursor;
BEGIN
    -- Apertura explícita del refcursor
    OPEN v_cur_ref FOR
        SELECT id, nombre, salario, departamento
        FROM tienda.empleados
        WHERE salario >= p_salario_min
        ORDER BY salario DESC;

    -- RETURN QUERY recupera las filas del cursor abierto
    RETURN QUERY
    FETCH ALL FROM v_cur_ref;

    CLOSE v_cur_ref;
END;
$$ LANGUAGE plpgsql;

-- Ejecución:
SELECT * FROM tienda.fn_empleados_altos_salarios_rq(5000.00);
```

Resultado:

v_id	v_nombre	v_salario	v_departamento
5	James Smith	7000.00	Dirección
7	Agnes Miller	6100.00	IT
3	Carla Gómez	5200.00	IT

Ejemplo 4.4: RETURN QUERY combinado con SETOF RECORD y %ROWTYPE

Enunciado: Devuelve un conjunto de registros SETOF RECORD utilizando una variable basada en %ROWTYPE filtrando los productos activos de la tienda.

```

CREATE OR REPLACE FUNCTION tienda.fn_productos_activos_rq()
RETURNS SETOF RECORD AS $$
DECLARE
    -- Referencia al tipo de fila completo de la tabla productos
    v_prod_dummy tienda.productos%ROWTYPE;
BEGIN
    RETURN QUERY
    SELECT producto_id, nombre, precio, stock
    FROM tienda.productos
    WHERE activo = TRUE;
END;
$$ LANGUAGE plpgsql;
```

```
-- Ejecución:
SELECT * FROM tienda.fn_productos_activos_rq()
AS (
  v_producto_id INTEGER,
  v_nombre TEXT,
  v_precio NUMERIC(10,2),
  v_stock INTEGER
);
```

Resultado:

v_producto_id	v_nombre	v_precio	v_stock
1	Teclado mecánico	79.90	50
2	Ratón inalámbrico	39.50	100
3	Monitor 27"	249.00	20

Tabla Comparativa de Sintaxis de Retorno

Tipo de Retorno	Ventajas	Inconvenientes
RETURNS TABLE(...)	Declara explícitamente los nombres y tipos de las columnas devueltas. No requiere especificar la estructura al llamar la función.	Debe coincidir exactamente con el orden y tipo de las variables asignadas.
RETURNS SETOF RECORD	Alta flexibilidad cuando la estructura de columnas cambia dinámicamente o se reutiliza la función.	Exige definir la cláusula AS (...) explícitamente en la consulta SELECT invocadora.
RETURN QUERY	No requiere bucles LOOP explícitos; PostgreSQL optimiza el almacenamiento en búfer de resultados.	No permite transformar o procesar lógica fila por fila dentro del cuerpo del bucle.
%ROWTYPE	Garantiza la integridad del tipo si la estructura subyacente de la tabla cambia.	Ocupa en memoria todos los campos de la tabla aunque solo se utilicen unos pocos.

User prompt: en RETURN NEXT (v_id, v_nombre, v_salario, v_departamento); podrias haber hecho solamente, return next v_emp_rec. no ? si es asi pon algun ejmplo

Response: **Sí, absolutamente.** De hecho, cuando se trabaja con variables de tipo %ROWTYPE o RECORD completo, escribir RETURN NEXT v_emp_rec; es la forma más directa, limpia e idiomática de hacerlo en PL/pgSQL, ya que evita tener que desestructurar el registro campo por campo antes de devolverlo.

A continuación tienes los ejemplos prácticos aplicados a ambos contextos: RETURNS TABLE y RETURNS SETOF RECORD.

1. Ejemplo con %ROWTYPE y RETURNS SETOF RECORD (Directo)

Enunciado: Recorrer los empleados con un cursor explícito cargando la fila entera en una variable %ROWTYPE y devolver la estructura completa usando RETURN NEXT v_emp_rec;.

```
CREATE OR REPLACE FUNCTION tienda.fn_empleados_rowtype_directo(IN p_dep TEXT)
RETURNS SETOF RECORD AS $$
DECLARE
  v_cur_emp CURSOR FOR
    SELECT *
    FROM tienda.empleados
    WHERE departamento = p_dep;

  -- Variable de tipo fila completa de la tabla empleados
  v_emp_rec tienda.empleados%ROWTYPE;
BEGIN
  OPEN v_cur_emp;
  LOOP
    FETCH v_cur_emp INTO v_emp_rec;
    EXIT WHEN NOT FOUND;

    -- Retorna directamente el objeto de fila completo sin desestructurar
    RETURN NEXT v_emp_rec;
  END LOOP;
  CLOSE v_cur_emp;
END;
$$ LANGUAGE plpgsql;

-- Ejecución (al devolver SETOF RECORD, definimos la estructura esperada):
SELECT * FROM tienda.fn_empleados_rowtype_directo('IT')
AS (
  v_id INTEGER,
  v_nombre TEXT,
```

```

v_salario NUMERIC(10,2),
v_departamento TEXT,
v_fecha_contratacion DATE
);

```

Resultado:

v_id	v_nombre	v_salario	v_departamento	v_fecha_contratacion
2	Luis Martínez	4500.00	IT	2026-03-20
3	Carla Gómez	5200.00	IT	2026-03-20
7	Agnes Miller	6100.00	IT	2026-03-20

2. Ejemplo con RECORD y RETURNS SETOF tienda.empleados

Si deseas retornar filas completas sin tener que escribir la cláusula AS (...) al hacer la llamada SELECT, puedes definir la función con RETURNS SETOF nombre_tabla o RETURNS SETOF %ROWTYPE. En ese caso, RETURN NEXT v_rec; funciona de forma nativa.

Enunciado: Consultar los productos activos con un cursor implícito almacenado en un RECORD y devolver las filas directamente.

```

CREATE OR REPLACE FUNCTION tienda.fn_productos_completos_directo()
RETURNS SETOF tienda.productos AS $$
DECLARE
    v_rec RECORD;
BEGIN
    FOR v_rec IN
        SELECT *
        FROM tienda.productos
        WHERE activo = TRUE
    LOOP
        -- Devuelve el registro completo directamente
        RETURN NEXT v_rec;
    END LOOP;
END;
$$ LANGUAGE plpgsql;

```

```

-- Ejecución (no requiere la cláusula AS):
SELECT * FROM tienda.fn_productos_completos_directo();

```

Resultado:

producto_id	nombre	precio	stock	activo
1	Teclado mecánico	79.90	50	t
2	Ratón inalámbrico	39.50	100	t
3	Monitor 27"	249.00	20	t

3. Ejemplo con RETURNS TABLE y Asignación Directa (INTO)

Si usas RETURNS TABLE, PostgreSQL vincula automáticamente las variables definidas en la cabecera. Por lo tanto, puedes hacer la lectura del cursor directamente sobre el conjunto de variables de retorno sin declarar variables intermedias ni usar desestructuración:

```

CREATE OR REPLACE FUNCTION tienda.fn_clientes_directo_table()
RETURNS TABLE (
    v_cliente_id INTEGER,
    v_nombre TEXT,
    v_email TEXT,
    v_fecha_alta DATE
) AS $$
DECLARE
    v_cur_clientes CURSOR FOR SELECT * FROM tienda.clientes;
BEGIN
    OPEN v_cur_clientes;
    LOOP
        -- Se vuelca el resultado del cursor directamente en las variables de salida (RETURNS TABLE)
        FETCH v_cur_clientes INTO v_cliente_id, v_nombre, v_email, v_fecha_alta;
        EXIT WHEN NOT FOUND;

        -- Al invocar RETURN NEXT sin parámetros, se devuelven los valores actuales de v_cliente_id, v_nom
        RETURN NEXT;
    END LOOP;
    CLOSE v_cur_clientes;
END;
$$ LANGUAGE plpgsql;

-- Ejecución:
SELECT * FROM tienda.fn_clientes_directo_table();

```

Resultado:

v_cliente_id	v_nombre	v_email	v_fecha_alta
1	Juan Pérez	juan@example.com	2026-03-20
2	María López	maria@example.com	2026-03-20

¿Cuándo conviene desestructurar y cuándo usar RETURN NEXT v_rec?

- Usa **RETURN NEXT v_rec**; o **RETURN NEXT v_emp_rec**; cuando devuelvas **todas** las columnas del origen tal cual están definidas, sin aplicar transformaciones ni cálculos intermedios.
- **Desestructura variable por variable (v_id := ...)** únicamente si necesitas modificar/filtrar campos al vuelo (por ejemplo, aplicar un descuento a v_precio, concatenar un texto a v_nombre o calcular un subtotal).

User prompt: ahora haz ejemplo con variables de salida o registros transformados.

Response: A continuación verás varios ejemplos prácticos en los que **modificamos, calculamos o transformamos los datos** dentro del bucle del cursor antes de devolverlos, utilizando tanto **variables de salida (OUT / RETURNS TABLE)** como **registros procesados (RECORD / %ROWTYPE)**.

1. Transformación con RETURNS TABLE y Cursor Implícito

Enunciado: Aplica un descuento del 15% al precio de cada producto y genera un diagnóstico de inventario ("Reposición urgente" si el stock es menor a 30, o "Stock adecuado" en caso contrario).

```
CREATE OR REPLACE FUNCTION tienda.fn_productos_descuento()
RETURNS TABLE (
    v_producto_id INTEGER,
    v_nombre TEXT,
    v_precio_original NUMERIC(10,2),
    v_precio_rebajado NUMERIC(10,2),
    v_estado_stock TEXT
) AS $$
DECLARE
    v_rec RECORD;
BEGIN
    FOR v_rec IN
        SELECT producto_id, nombre, precio, stock
        FROM tienda.productos
    LOOP
        -- Asignación y transformación de campos a las variables de salida
        v_producto_id := v_rec.producto_id;
        v_nombre := UPPER(v_rec.nombre); -- Transformación de texto
        v_precio_original := v_rec.precio;
        v_precio_rebajado := ROUND(v_rec.precio * 0.85, 2); -- Transformación numérica (-15%)

        -- Lógica condicional sobre el registro
        IF v_rec.stock < 30 THEN
            v_estado_stock := 'Reposición urgente (Quedan ' || v_rec.stock || ')';
        ELSE
            v_estado_stock := 'Stock adecuado (' || v_rec.stock || ' unidades)';
        END IF;

        RETURN NEXT; -- Devuelve las variables declaradas en RETURNS TABLE
    END LOOP;
END;
$$ LANGUAGE plpgsql;

-- Ejecución:
SELECT * FROM tienda.fn_productos_descuento();
```

Resultado:

v_producto_id	v_nombre	v_precio_original	v_precio_rebajado	v_estado_stock
1	TECLADO MECÁNICO	79.90	67.92	Stock adecuado (50 unidades)
2	RATÓN INALÁMBRICO	39.50	33.58	Stock adecuado (100 unidades)
3	MONITOR 27"	249.00	211.65	Reposición urgente (Quedan 20)

2. Transformación de %ROWTYPE devuelta en RETURNS SETOF RECORD

Enunciado: Consulta los datos de los empleados mediante un cursor explícito cargado en %ROWTYPE, aplica un incremento de salario dinámico según el departamento y devuelve los campos modificados en un SETOF RECORD.

```
CREATE OR REPLACE FUNCTION tienda.fn_ajuste_salarial_emp()
RETURNS SETOF RECORD AS $$
DECLARE
    v_cur_emp CURSOR FOR SELECT * FROM tienda.empleados;
```

```

v_emp_rec tienda.empleados%ROWTYPE;

-- Variables transformadas para la salida
v_id INTEGER;
v_nombre TEXT;
v_salario_antiguo NUMERIC(10,2);
v_salario_nuevo NUMERIC(10,2);
v_departamento TEXT;
BEGIN
  OPEN v_cur_emp;
  LOOP
    FETCH v_cur_emp INTO v_emp_rec;
    EXIT WHEN NOT FOUND;

    v_id          := v_emp_rec.id;
    v_nombre      := v_emp_rec.nombre;
    v_salario_antiguo := v_emp_rec.salario;
    v_departamento := v_emp_rec.departamento;

    -- Lógica de transformación del salario según el departamento
    IF v_emp_rec.departamento = 'IT' THEN
      v_salario_nuevo := v_emp_rec.salario * 1.10; -- +10%
    ELSIF v_emp_rec.departamento = 'Ventas' THEN
      v_salario_nuevo := v_emp_rec.salario * 1.05; -- +5%
    ELSE
      v_salario_nuevo := v_emp_rec.salario * 1.02; -- +2%
    END IF;

    RETURN NEXT (v_id, v_nombre, v_salario_antiguo, ROUND(v_salario_nuevo, 2), v_departamento);
  END LOOP;
  CLOSE v_cur_emp;
END;
$$ LANGUAGE plpgsql;

-- Ejecución:
SELECT * FROM tienda.fn_ajuste_salarial_emp()
AS (
  v_id INTEGER,
  v_nombre TEXT,
  v_salario_antiguo NUMERIC(10,2),
  v_salario_nuevo NUMERIC(10,2),
  v_departamento TEXT
);

```

Resultado:

v_id	v_nombre	v_salario_antiguo	v_salario_nuevo	v_departamento
1	Ana Torres	2800.00	2940.00	Ventas
2	Luis Martínez	4500.00	4950.00	IT
3	Carla Gómez	5200.00	5720.00	IT
4	Dmitry Komarov	3100.00	3162.00	Contabilidad
5	James Smith	7000.00	7140.00	Dirección
6	Mary Sexton	3900.00	3978.00	Marketing
7	Agnes Miller	6100.00	6710.00	IT
8	Chun Yen	2950.00	3097.50	Ventas

3. Modificación del mismo tipo %ROWTYPE antes de devolverlo

También puedes modificar directamente los campos de la variable %ROWTYPE dentro del bucle y retornar la estructura entera con RETURN NEXT v_emp_rec;.

Enunciado: Recorre la tabla de empleados, oculta el apellido por privacidad (anonimización de datos) y retorna la tabla de tipo %ROWTYPE modificada.

```

CREATE OR REPLACE FUNCTION tienda.fn_empleados_anonimizados()
RETURNS SETOF tienda.empleados AS $$
DECLARE
  v_emp_rec tienda.empleados%ROWTYPE;
BEGIN
  FOR v_emp_rec IN SELECT * FROM tienda.empleados LOOP
    -- Modificamos la variable %ROWTYPE en el aire
    v_emp_rec.nombre := SPLIT_PART(v_emp_rec.nombre, ' ', 1) || ' ***';

    -- Si el salario supera los 5000, le marcamos un indicativo
    IF v_emp_rec.salario > 5000.00 THEN
      v_emp_rec.departamento := COALESCE(v_emp_rec.departamento, '') || ' [SENIOR]';
    END IF;
  END LOOP;
END;

```

```

-- Retornamos el registro %ROWTYPE alterado
RETURN NEXT v_emp_rec;
END LOOP;
END;
$$ LANGUAGE plpgsql;

-- Ejecución:
SELECT * FROM tienda.fn_empleados_anonimizados();

```

Resultado:

id	nombre	salario	departamento	fecha_contratacion
1	Ana ***	2800.00	Ventas	2026-03-20
2	Luis ***	4500.00	IT	2026-03-20
3	Carla ***	5200.00	IT [SENIOR]	2026-03-20
4	Dmitry ***	3100.00	Contabilidad	2026-03-20
5	James ***	7000.00	Dirección [SENIOR]	2026-03-20
6	Mary ***	3900.00	Marketing	2026-03-20
7	Agnes ***	6100.00	IT [SENIOR]	2026-03-20
8	Chun ***	2950.00	Ventas	2026-03-20

4. Agregación y Transformación con Parámetros OUT Múltiples

Enunciado: Procesar las líneas del pedido con `pedido_id = 1` sumando su importe total, contando las unidades y construyendo un resumen descriptivo en formato JSON mediante un cursor explícito.

```

CREATE OR REPLACE FUNCTION tienda.fn_resumen_pedido_transformado(
    IN v_pedido_id INTEGER,
    OUT v_total_pagar NUMERIC(10,2),
    OUT v_unidades_totales INTEGER,
    OUT v_resumen_textual TEXT
) AS $$
DECLARE
    v_cur_lineas CURSOR FOR
        SELECT lp.cantidad, lp.precio_unit, p.nombre
        FROM tienda.lineas_pedido lp
        JOIN tienda.productos p ON lp.producto_id = p.producto_id
        WHERE lp.pedido_id = v_pedido_id;

    v_rec RECORD;
    v_lista_productos TEXT := '';
BEGIN
    v_total_pagar := 0;
    v_unidades_totales := 0;

    OPEN v_cur_lineas;
    LOOP
        FETCH v_cur_lineas INTO v_rec;
        EXIT WHEN NOT FOUND;

        -- Transformaciones y acumuladores
        v_total_pagar := v_total_pagar + (v_rec.cantidad * v_rec.precio_unit);
        v_unidades_totales := v_unidades_totales + v_rec.cantidad;

        -- Concatenación de productos para el texto final
        v_lista_productos := v_lista_productos || v_rec.cantidad || 'x ' || v_rec.nombre || '; ';
    END LOOP;
    CLOSE v_cur_lineas;

    v_resumen_textual := 'Pedido #' || v_pedido_id || ' contiene: ' || RTRIM(v_lista_productos, '; ');
END;
$$ LANGUAGE plpgsql;

-- Ejecución:
SELECT * FROM tienda.fn_resumen_pedido_transformado(1);

```

Resultado:

v_total_pagar	v_unidades_totales	v_resumen_textual
158.90	3	Pedido #1 contiene: 1x Teclado mecánico; 2x Ratón inalámbrico